

CONSUMER LOAN PREDICTION USING MACHINE LEARNING

Team Members:

Dogga Swathi(2448850148)

Thota Durga Bhavani(2448850040)

Pagadala Jayasree(2448850132)

Dasari Sailaja(2448850136)

Course: B.Sc Artificial Intelligence

Date:03-02-2026

Abstract

This project presents a **Consumer Loan Prediction System using Machine Learning** to identify customers who are likely to purchase a consumer loan. Customer financial and demographic data are preprocessed and used to train models such as Logistic Regression, Decision Tree, KNN, and Random Forest. These models are evaluated using Precision, Recall, F1-Score, and ROC-AUC, and **Random Forest** is selected as the best model. The final model is deployed using a Flask backend and integrated with an HTML-based user interface to provide real-time loan prediction results, helping banks improve decision making and marketing efficiency.

1. Introduction

Loan approval is a crucial process in financial institutions. Manual verification of customer details is time-consuming and prone to human error. With the growth of data and technology, machine learning can automate this process efficiently. This project introduces a Consumer Loan Prediction using Machine Learning to predict whether a customer is eligible for a loan based on historical data. The system improves speed, accuracy, and consistency in decision-making.

2. Problem Statement

Banks run marketing campaigns to promote consumer loans, but identifying the right customers manually is difficult and costly. Many campaigns target users who are not interested in taking loans, which reduces conversion rates. There is a need for an automated system to analyze customer financial and demographic data. This project uses machine learning to predict customers who are likely to purchase a consumer loan. The system helps banks improve efficiency and reduce campaign cost.

3. Objectives

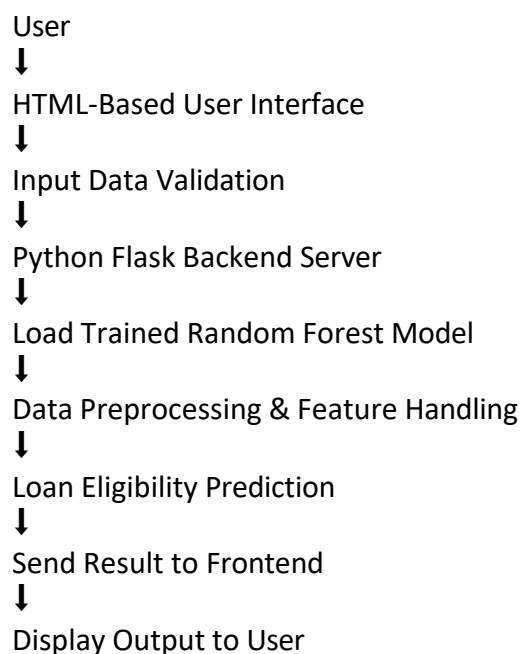
1. To collect and preprocess consumer loan data.
 2. To analyze and clean the dataset.
 3. To train multiple machine learning models.
 4. To evaluate model performance using metrics.
 5. To select the best performing model.
 6. To integrate the model with a web-based user interface.
 7. To display prediction results to users.
-

4. System Architecture

The system consists of three major components:

1. User Interface (HTML/CSS)
2. Backend Server (Python with Flask)
3. Machine Learning Model (Random Forest)

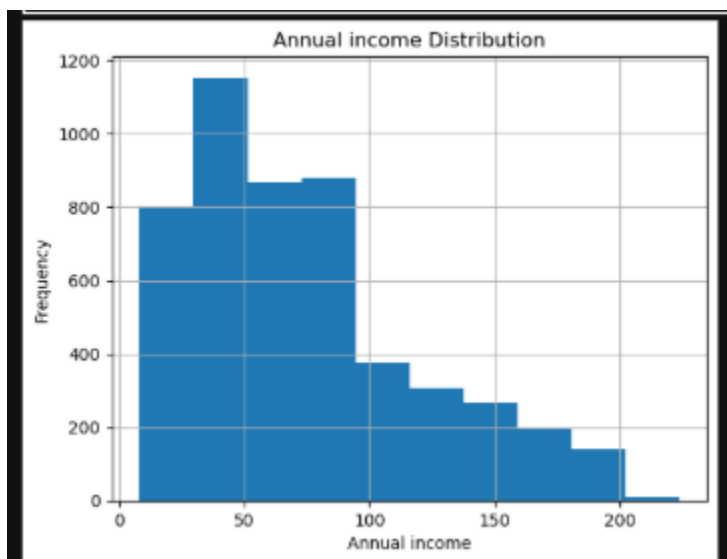
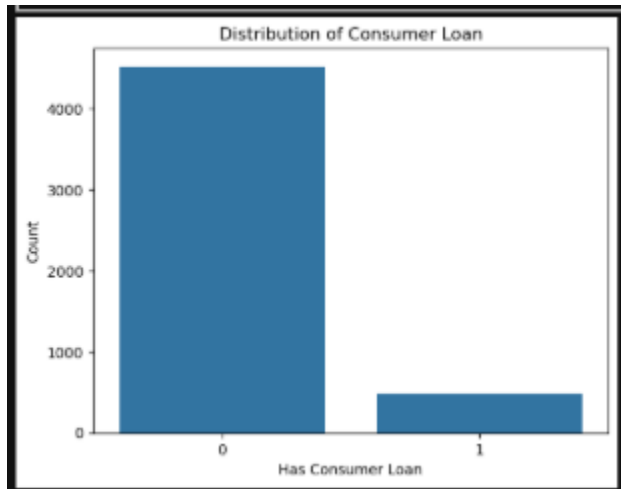
The user enters details in the web interface. The data is sent to the backend server. The trained machine learning model processes the input and predicts loan eligibility. The result is returned to the interface and displayed to the user.



5. Dataset Description

The Consumer Loan Prediction System uses the **Customer_Financial_Info** dataset, which contains financial and demographic details of bank customers. Each record represents a customer profile with attributes such as **Age, Years of Experience, Annual Income, ZIP Code, Family Size, Average Spending, Education Level, Mortgage Amount, Securities Account Status, CD Account Status, Online Banking Usage, Credit Card Ownership**, and other financial indicators.

The target variable in the dataset is **Has_Consumer_Loan**, which indicates whether a customer has taken a consumer loan (1) or not (0). This dataset is used to train machine learning models to learn patterns from customer behavior and predict potential loan customers accurately.



A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Customer	Age	Years_Exp	Annual_Inc	ZIP_Code	Family_size	Avg_Spend	Education	Mortgage	Has_Consum	Has_Secur	Has_CD_A	Uses_Onlin	Has_CreditCard	
1	25	1	49	91107	4	1.6	1	0	0	1	0	0	0	
2	45	19	34	90089	3	1.5	1	0	0	1	0	0	0	
3	39	15	11	94720	1	1	1	0	0	0	0	0	0	
4	35	9	100	94112	1	2.7	2	0	0	0	0	0	0	
5	35	8	45	91330	4	1	2	0	0	0	0	0	1	
6	37	13	29	92121	4	0.4	2	155	0	0	0	1	0	
7	53	27	72	91711	2	1.5	2	0	0	0	0	1	0	
8	50	24	22	93943	1	0.3	3	0	0	0	0	0	1	
9	35	10	81	90089	3	0.6	2	104	0	0	0	1	0	
10	34	9	180	93023	1	8.9	3	0	1	0	0	0	0	
11	65	39	105	94710	4	2.4	3	0	0	0	0	0	0	
12	29	5	45	90277	3	0.1	2	0	0	0	0	1	0	
13	48	23	114	93106	2	3.8	3	0	0	1	0	0	0	
14	59	32	40	94920	4	2.5	2	0	0	0	0	1	0	
15	67	41	112	91741	1	2	1	0	0	1	0	0	0	

6. Data Preprocessing

Data preprocessing involves handling missing values, encoding categorical variables, and scaling numerical features. Log transformation is applied to reduce skewness in income data. Standardization ensures that all numerical features are on the same scale, improving the performance of machine learning models.

```
[7]: # Remove multiple columns, e.g., ZIP_Code and Customer_ID
data = data.drop(["ZIP_Code", "Customer_ID"], axis=1)
```

```
[9]: data.columns = data.columns.str.strip()
```

```
[11]: print(data.columns)
```

```
Index(['Age', 'Years_Experience', 'Annual_Income', 'Family_size',
       'Avg_Spending', 'Education_Level', 'Mortgage', 'Has_Consumer_Loan',
       'Has_Securities_Account', 'Has_CD_Account', 'Uses_Online_Banking',
       'Has_CreditCard'],
      dtype='object')
```

```
[30]: # to check null values
data.isnull().sum()
```

```
[30]: Age          0
Years_Experience  0
Annual_Income    0
Family_size      0
Avg_Spending     0
Education_Level  0
Mortgage         0
Has_Consumer_Loan 0
Has_Securities_Account 0
Has_CD_Account   0
Uses_Online_Banking 0
Has_CreditCard   0
dtype: int64
```

```
[32]: #to check duplicate values
data.drop_duplicates(inplace=True)
data
```

```
[32]:
```

	Age	Years_Experience	Annual_Income	Family_size	Avg_Spending	Education_Level	Mortgage	Has_Consumer_Loan	Has_Securities_Account	Has_CD_Account	Uses_Online_Banking	Has_CreditCard
0	25	1	49	4	1.6	1	0	0	1	0	0	0
1	45	19	34	3	1.5	1	0	0	1	0	0	0
2	39	15	11	1	1.0	1	0	0	0	0	0	0
3	35	9	100	1	2.7	2	0	0	0	0	0	0
4	35	8	45	4	1.0	2	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
4995	29	3	40	1	1.9	3	0	0	0	0	0	0
4996	30	4	15	4	0.4	1	85	0	0	0	0	0

7. Machine Learning Models Used

The following machine learning models are implemented and compared: -

- Logistic Regression
- Decision Tree
- K-Nearest Neighbors (KNN)
- Random Forest

Each model is evaluated using accuracy, precision, recall, F1-score, and ROC-AUC score. Based on comparison, Random Forest gives the best performance and is selected for final prediction.

```
[70]: #Random Forest (Ensemble)
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(x_train, y_train)

y_pred_rf = rf_model.predict(x_test)
rf_model

[70]: RandomForestClassifier
RandomForestClassifier(random_state=42)

[84]: # random forest

# Corrected hyperparameter grid
param_grid_rf = {
    'n_estimators': [50, 100],          # number of trees
    'max_depth': [5, 10, None],         # max depth of each tree
    'min_samples_split': [2, 5],        # minimum samples to split a node
    'min_samples_leaf': [1, 2],         # minimum samples at a leaf
    'max_features': ['sqrt', 'log2', None] # corrected values
}

# GridSearchCV with progress and parallel processing
grid_rf = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid=param_grid_rf,
    cv=5,
    verbose=1, # shows progress
    n_jobs=-1 # use all CPU cores
)

# Fit grid search
grid_rf.fit(x_train, y_train)

# Best hyperparameters
print("Best Random Forest Parameters:", grid_rf.best_params_)

# Use best model
best_rf = grid_rf.best_estimator_

Fitting 5 folds for each of 72 candidates, totalling 360 fits
Best Random Forest Parameters: {'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
```

8. Model Training and Evaluation

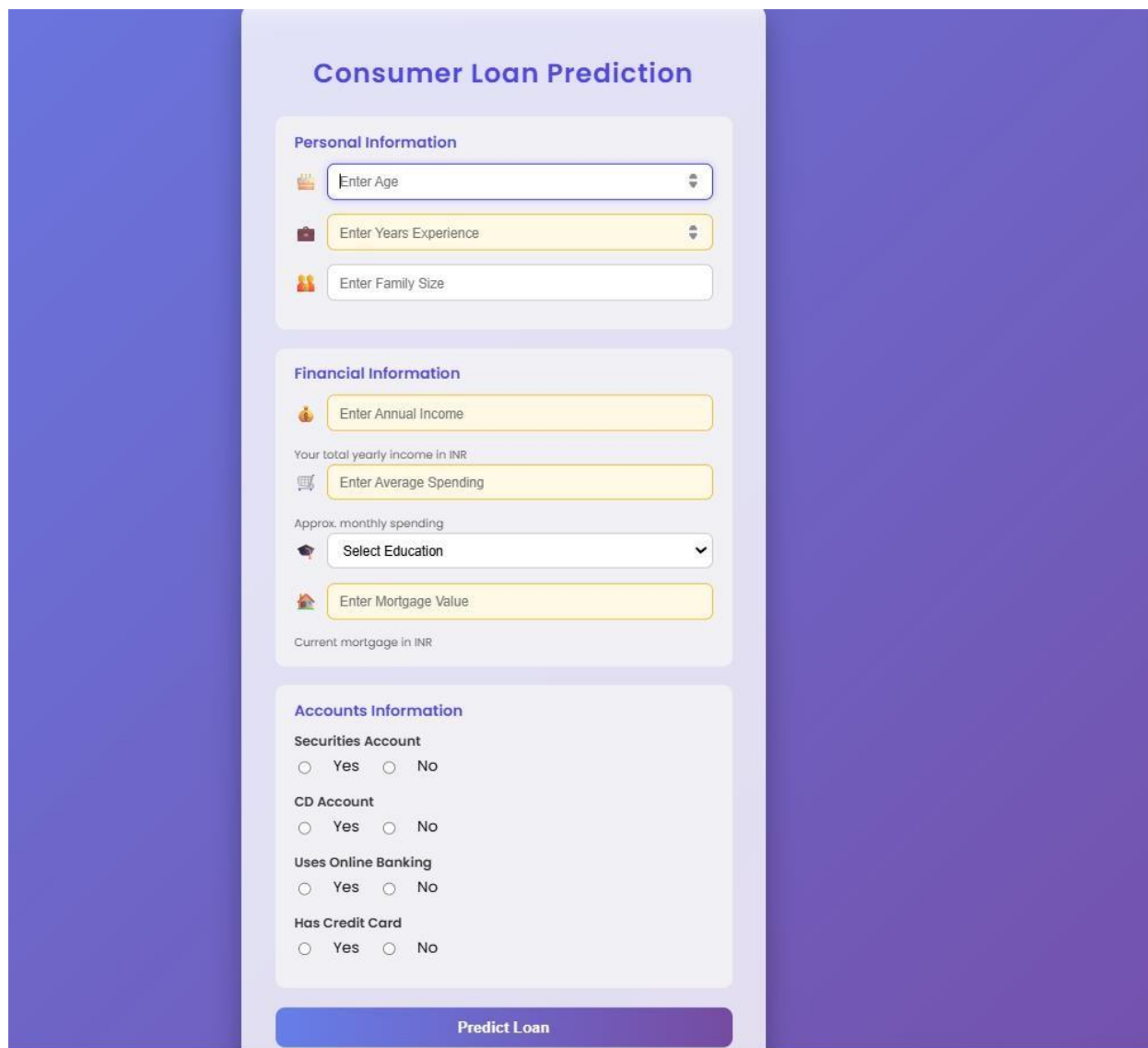
The dataset is split into training and testing sets. Models are trained on the training data and tested on unseen data. Evaluation metrics and ROC curves are used to compare performance. Random Forest provides better generalization and higher accuracy compared to other models.

9. Model Deployment and Integration

After training, the best model is saved using the Joblib library. A Flask backend loads the saved model and connects it with the HTML interface. When the user submits the form, the backend processes the input, performs prediction, and returns the result to the frontend for display.

10. User Interface Design

The user interface is developed using HTML and CSS. It provides input fields for customer information such as age, income, employment experience, home ownership, loan amount, interest rate, and credit score. After submitting the form, the prediction result is displayed clearly to the user.



The image shows a web form titled "Consumer Loan Prediction" with a purple gradient background. The form is divided into three main sections: Personal Information, Financial Information, and Accounts Information. Each section contains several input fields and a "Predict Loan" button at the bottom.

Consumer Loan Prediction

Personal Information

- Enter Age
- Enter Years Experience
- Enter Family Size

Financial Information

- Enter Annual Income
- Your total yearly income in INR
- Enter Average Spending
- Approx. monthly spending
- Select Education
- Enter Mortgage Value
- Current mortgage in INR

Accounts Information

- Securities Account**
☐ Yes ☐ No
- CD Account**
☐ Yes ☐ No
- Uses Online Banking**
☐ Yes ☐ No
- Has Credit Card**
☐ Yes ☐ No

Predict Loan

11. Results

The system successfully predicts whether a consumer is eligible for a loan. The Random Forest model produces reliable and accurate results. The prediction output is shown instantly on the web interface, making the system interactive and efficient.

Loan Approved  (39.00% chance)

[Go Back](#)

Loan Not Approved  (0.00% chance)

[Go Back](#)

12. Conclusion

The Consumer Loan Prediction System demonstrates the effective use of machine learning for automating loan eligibility decisions. By integrating a trained Random Forest model with a web-based interface, the system provides fast and accurate predictions. This project highlights the practical application of artificial intelligence in financial services.
